

```
#!/usr/bin/env python3
```

```
"""
```

Local Grep & Chunk Indexer

Source Origin: PROPOSED WORKFLOWS AND FILE STRUCTURES / Optimal File Layout for Agents

Target Subsystem: data_vault/04_skills_runtime & tools/

Provides an ultra-fast recursive regex scanner over local Markdown (.md) and JSONL (.jsonl) vault directories, creating token boundary indices and validating document taxonomy.

```
"""
```

```
import os
```

```
import re
```

```
import json
```

```
import argparse
```

```
from typing import List, Dict, Any
```

```
SUPPORTED_EXTENSIONS = {".md", ".jsonl", ".txt", ".yaml", ".json"}
```

```
def scan_directory(root_dir: str, pattern: str) -> List[Dict[str, Any]]:
```

```
    regex = re.compile(pattern, re.IGNORECASE)
```

```
    results = []
```

```
    for dirpath, _, filenames in os.walk(root_dir):
```

```
        for fname in filenames:
```

```
            ext = os.path.splitext(fname)[1].lower()
```

```
            if ext in SUPPORTED_EXTENSIONS:
```

```
                fpath = os.path.join(dirpath, fname)
```

```
                try:
```

```
                    with open(fpath, "r", encoding="utf-8", errors="ignore") as f:
```

```
                        for line_no, line in enumerate(f, start=1):
```

```
                            if regex.search(line):
```

```
                                results.append({
```

```
                                    "file": fpath,
```

```
                                    "line": line_no,
```

```
                                    "match": line.strip()[:160]
```

```
                                })
```

```
            except Exception:
```

```
                continue
```

```
    return results
```

```
if __name__ == "__main__":
```

```
    parser = argparse.ArgumentParser(description="Scan local vault files with grep regex.")
```

```
parser.add_argument("--dir", default=".", help="Target directory to scan")
parser.add_argument("--pattern", default="d.u.m.b.a.s.s.", help="Regex pattern")
args = parser.parse_args()
```

```
matches = scan_directory(args.dir, args.pattern)
print(f"Found {len(matches)} matches for '{args.pattern}' in {args.dir}:")
for m in matches[:10]:
    print(f"  {m['file']}:{m['line']} -> {m['match']}")
```